

Technische Komplettdokumentation:

Modularer Schicht- & Haushaltsplaner 2.0

System-Zusammenfassung & Release-Stand

Dieses Dokument bündelt alle für die Einrichtung und den Betrieb des modularen Schichtplaners erforderlichen Komponenten. Es schließt die Lücken bezüglich der vollständigen Helferstrukturen, sämtlicher YAML-Skripte und -Automatisierungen, der Lovelace-Dashboard-Konfigurationen und bietet eine lückenlose Schritt-für-Schritt-Installationsanleitung.

1. Komplette Helferliste (Backend-Entitäten)

Die folgenden Helfer (Entities) müssen im Home Assistant Backend angelegt werden. Es wird dringend empfohlen, die IDs exakt wie vorgegeben (umlautfrei) zu benennen, um Syntaxfehler in den Skripten zu vermeiden.

1.1 Eingabe-Helfer (UI-Steuerung)

Entity-ID	Typ	Optionen / Wertebereich	Beschreibung / Verwendungszweck
<code>input_select.planung_person</code>	Dropdown	Sascha, Sabine, Samuel, Simon, Familie	Hauptauswahl der zu planenden Person im Dashboard. Steuert die Sichtbarkeit der Schichten.
<code>input_select.planung_schichten_sascha</code>	Dropdown	Frühschicht, Spätschicht, Nachtschicht, Urlaub, Krank	Exklusive Schichtarten für Sascha. Reines UI-Eingabefeld.
<code>input_select.planung_schichten_sabine</code>	Dropdown	F2, F4, F5, Heurich, Urlaub, Krank	Exklusive Schichtarten für Sabine. Reines UI-Eingabefeld.
<code>input_select.schicht_auswahl</code>	Dropdown	Kiga ZU, Schule, Ausflug, Termin	Globale Termin- und Ereignisarten für die Kinder oder die gesamte Familie.
<code>input_datetime.ziel_datum</code>	Datum	Nur Datum (YYYY-MM-DD)	Das Startdatum des zu planenden Zeitraums.
<code>input_datetime.ziel_datum_ende</code>	Datum	Nur Datum (YYYY-MM-DD)	Das Enddatum (einschließlich) des zu planenden Zeitraums.
<code>input_datetime.start_zeit</code>	Uhrzeit	Nur Zeit (HH:MM:SS)	Optionale Startzeit für manuelle Termine (Standard: 00:00:00).
<code>input_datetime.end_zeit</code>	Uhrzeit	Nur Zeit (HH:MM:SS)	Optionale Endzeit für manuelle Termine (Standard: 00:00:00).

1.2 Statistik-Speicherzellen (Persistente Datenhaltung)

Diese Textfelder dienen als entkoppelte Datencontainer, um die Jahressummen direkt im Zustand zu speichern, wodurch rechenintensive Live-Abfragen im Dashboard entfallen.

Entity-ID	Typ	Standardwert	Beschreibung / Verwendungszweck
input_text.stats_sascha_fruehschicht	Text	0	Zähler für Saschas geleistete Frühschichten im aktuellen Jahr.
input_text.stats_sascha_spaetschicht	Text	0	Zähler für Saschas geleistete Spätschichten im aktuellen Jahr.
input_text.stats_sabine_heurich	Text	0	Zähler für Sabines geleistete Heurich-Dienste im aktuellen Jahr.

2. Komplette YAML-Dateien (Core-Logik)

Füge diese Codes in deine `scripts.yaml` bzw. `automations.yaml` ein oder erstelle sie direkt über die Home Assistant Benutzeroberfläche im erweiterten YAML-Modus.

2.1 Unter-Skript: Daten-Schreiber

script.schichtplaner_final (In `scripts.yaml` integrieren)

```

schichtplaner_final:
  alias: "Schichtplaner: Final"
  description: "Schreibt das berechnete Einzel-Event basierend auf der Person in den Zielkalender."
  fields:
    tag:
      description: "Das konkrete Datum für den Eintrag (YYYY-MM-DD)"
      example: "2026-05-15"
    person:
      description: "Name der ausgewählten Person"
      example: "Sascha"
    schicht:
      description: "Die Schicht- oder Ereignisbezeichnung"
      example: "Frühschicht"
    start_zeit:
      description: "Optionale Uhrzeit für den Beginn"
      example: "06:00:00"
    end_zeit:
      description: "Optionale Uhrzeit für das Ende"
      example: "14:30:00"
  sequence:
    - action: calendar.create_event
      target:
        entity_id: >-
          {% if person == 'Sascha' %} calendar.sascha
          {% elif person == 'Sabine' %} calendar.sabine
          {% else %} calendar.familie
          {% endif %}
      data:
        summary: "{{ schicht }}"
        start_date_time: "{{ tag }} {% if start_zeit != '' %} {{ start_zeit }} {% endif %}"
        end_date_time: "{{ tag }} {% if end_zeit != '' %} {{ end_zeit }} {% endif %}"

```

2.2 Haupt-Skript: Weiche & Schleifenberechnung

script.schichtplaner_zeitraum_planen (In scripts.yaml integrieren)

```
schichtplaner_zeitraum_planen:
  alias: "Schichtplaner: Zeitraum planen"
  description: "Errechnet den Zeitraum, löst das Geisterbuchungs-Problem und iteriert über die
Tage."
  sequence:
    - variables:
        t_start: "{{ states('input_datetime.ziel_datum') }}"
        t_end: "{{ states('input_datetime.ziel_datum_ende') }}"
        v_person: "{{ states('input_select.planung_person') }}"

        # Logische Weiche fängt unsichtbare UI-Zustände ab
        v_schicht: >-
            {% if v_person == 'Sascha' %}
                {{ states('input_select.planung_schichten_sascha') }}
            {% elif v_person == 'Sabine' %}
                {{ states('input_select.planung_schichten_sabine') }}
            {% else %}
                {{ states('input_select.schicht_auswahl') }}
            {% endif %}

        # Berechnet die Gesamtanzahl der Iterationstage (Inklusive Start- und Endtag)
        v_anzahl: >-
            {% set d1 = as_datetime(t_start) if as_datetime(t_start) is not none else now() %}
            {% set d2 = as_datetime(t_end) if as_datetime(t_end) is not none else d1 %}
            {{ ((d2.date() - d1.date()).days + 1) | int }}

    - repeat:
        count: "{{ v_anzahl }}"
        sequence:
            - action: script.schichtplaner_final
              data:
                tag: >-
                    {% set d = as_datetime(t_start) if as_datetime(t_start) is not none else now() %}
                    {{ (d + timedelta(days=repeat.index - 1)).strftime('%Y-%m-%d') }}
                person: "{{ v_person }}"
                schicht: "{{ v_schicht }}"
                start_zeit: "{{ states('input_datetime.start_zeit') }}"
                end_zeit: "{{ states('input_datetime.end_zeit') }}"

            - delay: "00:00:02" # I/O-Gedenksekunden zur Cache-Sicherung der SQLite-DB
            - action: script.schichtstatistik_aktualisieren
```

2.3 Statistik-Skript: Jahres-Aggregator

script.schichtstatistik_aktualisieren (In scripts.yaml integrieren)

```
schichtstatistik_aktualisieren:
  alias: "Schichtstatistik aktualisieren"
  description: "Liest das Gesamtjahr aus und schreibt die Summen in die Input-Texte."
  sequence:
    - action: calendar.get_events
      target:
        entity_id:
          - calendar.sascha
          - calendar.sabine
          - calendar.familie
      data:
        start_date_time: "2026-01-01T00:00:00"
        end_date_time: "2026-12-31T23:59:59"
      response_variable: cal

    - variables:
        s_ev: "{{ cal['calendar.sascha'].events if cal['calendar.sascha'] is defined else [] }}"
        b_ev: "{{ cal['calendar.sabine'].events if cal['calendar.sabine'] is defined else [] }}"

    - action: input_text.set_value
      target: { entity_id: input_text.stats_sascha_fruehschicht }
      data: { value: "{{ s_ev | selectattr('summary', 'search', 'Frühschicht') | list | length |
string }}" }

    - action: input_text.set_value
      target: { entity_id: input_text.stats_sascha_spaetschicht }
      data: { value: "{{ s_ev | selectattr('summary', 'search', 'Spätschicht') | list | length |
string }}" }

    - action: input_text.set_value
      target: { entity_id: input_text.stats_sabine_heurich }
      data: { value: "{{ b_ev | selectattr('summary', 'search', 'Heurich') | list | length |
string }}" }
```

2.4 Automatisierung: Täglicher Failover-Sync

In automations.yaml integrieren

```
- alias: "Schichtstatistik: Tägliche Systemsynchronisation"
  id: schichtstatistik_daily_update
  trigger:
    - platform: time
      at: "03:00:00"
    - platform: homeassistant
      event: start
  action:
    - action: script.schichtstatistik_aktualisieren
```

2.5 Haushalts-Skript: Autonomer 14-tägiger Müllplaner

script.muelltonnen_planer (In scripts.yaml integrieren)

```
muelltonnen_planer:
  alias: "Mülltonnen: Rhythmus planen (ganztägig)"
  description: "Generiert 14-tägige Mülltermine für das laufende Jahr 2026."
  sequence:
    - repeat:
        count: 52
        sequence:
          - action: calendar.create_event
            target:
              entity_id: calendar.familie
            data:
              summary: "Mülltonne"
              start_date: >-
                {{ (as_datetime('2026-01-11') + timedelta(days=(repeat.index - 1) *
14)).strftime('%Y-%m-%d') }}
              end_date: >-
                {{ (as_datetime('2026-01-12') + timedelta(days=(repeat.index - 1) *
14)).strftime('%Y-%m-%d') }}
```

3. Beispielkonfigurationen (Frontend Dashboard)

Füge diese YAML-Blöcke als Karten („Manuelle Karte“) in dein Lovelace-Dashboard ein.

3.1 Die dynamische Eingabemaske

Lovelace-Karte: Eingabe-Steuerung

```

type: vertical-stack
cards:
  - type: entities
    entities:
      - entity: input_select.planung_person
        name: "Wer wird geplant?"

  - type: conditional
    conditions:
      - entity: input_select.planung_person
        state: "Sascha"
    card:
      type: entities
      entities:
        - entity: input_select.planung_schichten_sascha
          name: "Sascha - Schichtauswahl"

  - type: conditional
    conditions:
      - entity: input_select.planung_person
        state: "Sabine"
    card:
      type: entities
      entities:
        - entity: input_select.planung_schichten_sabine
          name: "Sabine - Schichtauswahl"

  - type: conditional
    conditions:
      - condition: or
        conditions:
          - entity: input_select.planung_person
            state: "Samuel"
          - entity: input_select.planung_person
            state: "Simon"
          - entity: input_select.planung_person
            state: "Familie"
    card:
      type: entities
      entities:
        - entity: input_select.schicht_auswahl
          name: "Termin-Art / Ereignis"

  - type: entities
    entities:
      - entity: input_datetime.ziel_datum
        name: "Startdatum"
      - entity: input_datetime.ziel_datum_ende
        name: "Enddatum (einschließlich)"
      - entity: input_datetime.start_zeit
        name: "Optionale Startzeit"
      - entity: input_datetime.end_zeit
        name: "Optionale Endzeit"

  - type: button
    name: "Planungszeitraum in Kalender eintragen"
    icon: mdi:calendar-check

```

```
tap_action:
  action: call-service
  service: script.schichtplaner_zeitraum_planen
```

3.2 Die ressourcenschonende Statistik-Tabelle

Lovelace-Karte: Markdown-Statistik

```
type: markdown
title: " Schichtstatistik Gesamtjahr 2026"
content: >
  | Schichtart | Sascha | Sabine | |
|---|---|---|---|
  |  Frühschicht | {{ states('input_text.stats_sascha_fruehschicht') | default('0') }} | - |
  |  Spätschicht | {{ states('input_text.stats_sascha_spaetschicht') | default('0') }} | - |
  |  Dienst Heurich | - | {{ states('input_text.stats_sabine_heurich') | default('0') }} |
```


4. Schritt-für-Schritt Setup-Anleitung

Folge dieser exakten Reihenfolge bei der Einrichtung, um Fehler durch nicht aufgelöste Entitäten-Abhängigkeiten im System auszuschließen.

1. Schritt 1: Kalender-Instanzen deklarieren

Gehe in Home Assistant auf *Einstellungen* → *Geräte & Dienste* → *Integrationen*. Füge eine Kalender-Integration hinzu (z. B. Lokaler Kalender) und erstelle exakt drei Kalender mit folgenden Namen (die Entity-ID erzeugt sich automatisch):

- Name: `Sascha` → erzeugt `calendar.sascha`
- Name: `Sabine` → erzeugt `calendar.sabine`
- Name: `Familie` → erzeugt `calendar.familie`

2. Schritt 2: Backend-Helfer (Entities) anlegen

Gehe auf *Einstellungen* → *Geräte & Dienste* → *Helfer*. Erstelle über die Schaltfläche "+ Helfer erstellen" alle in **Kapitel 1** aufgeführten Dropdowns (`input_select`), Datumsfelder (`input_datetime`) und Textfelder (`input_text`). Achte penibel darauf, die Bezeichner exakt umlautfrei zu wählen (z.B. `stats_sascha_fruehschicht`).

3. Schritt 3: Das Unter-Skript (Daten-Schreiber) einspielen

Gehe auf *Einstellungen* → *Automatisierungen & Szenen* → *Skripte*. Erstelle ein neues Skript, wechsele oben rechts über die drei Punkte in den "In YAML bearbeiten"-Modus und füge den Code aus **Kapitel 2.1** ein. Speichere das Skript ab. *Hinweis: Dies muss zuerst geschehen, da das Hauptskript sonst beim Speichern eine fehlende Abhängigkeit meldet!*

4. Schritt 4: Das Statistik-Skript einspielen

Erstelle ein weiteres Skript für den Jahres-Aggregator (siehe Code in **Kapitel 2.3**), schalte in den YAML-Modus, füge den Code ein und speichere.

5. Schritt 5: Das Haupt-Skript einspielen

Erstelle das übergeordnete Steuerungsskript (siehe Code in **Kapitel 2.2**), füge den Code im YAML-Modus ein und speichere es ab. Nun steht die Kernlogik.

6. Schritt 6: Failover-Automatisierung und Haushaltsplaner aktivieren

Füge die Synchronisations-Automatisierung (**Kapitel 2.4**) unter *Automatisierungen* hinzu. Erstelle zudem das Müllplaner-Skript (**Kapitel 2.5**). Führe das Müllplaner-Skript einmalig manuell aus (über "Ausführen" in den Skript-Optionen), um die 52 Jahrestermine für 2026 sofort in den Familienkalender zu schreiben.

7. Schritt 7: Dashboard-Schnittstelle aufbauen

Wechsle zu deinem Dashboard, klicke oben rechts auf "Dashboard bearbeiten" und füge eine neue Karte vom Typ "Manuell" hinzu. Ersetze den dortigen Platzhalter-Code vollständig durch das YAML aus **Kapitel 3.1**. Wiederhole den Vorgang für die Statistik-Tabelle aus **Kapitel 3.2**.

System erfolgreich einsatzbereit!

Sobald alle Schritte durchlaufen sind, ist das System vollständig operabel. Teste die Einrichtung, indem du für Sascha oder Sabine einen Zeitraum von 3 Tagen buchst. Die Kalender befüllen sich sequenziell und exakt 2 Sekunden nach Abschluss des Schreibvorgangs springt deine Lovelace-Statistik-Tabelle automatisch auf den neuen, inkrementierten Stand an.